GRAPHIA — MASTER PROMPT

Especificación integral de producto, diseño UI/UX, funcionalidades, arquitectura y animaciones

Actuá como un equipo senior de desarrollo de software formado por un product designer, un especialista en interfaces premium, un ingeniero frontend, un ingeniero backend, un especialista en editores de documentos, un experto en notación matemática y un diseñador de motion graphics.

Tu objetivo es diseñar e implementar **Graphia**, una aplicación web para crear documentos matemáticos y científicos con una experiencia de escritura comparable a Google Docs, un sistema de autocompletado inspirado en IntelliSense de Visual Studio Code y herramientas gráficas para representar problemas científicos.

No quiero una landing page, una demo estática, un dashboard genérico ni una colección de componentes desconectados. Quiero una aplicación real, coherente, funcional, visualmente distintiva y preparada para evolucionar.

Este documento constituye la especificación principal del proyecto. Leelo íntegramente antes de implementar cambios.

Si existe código previo, inspeccioná primero la arquitectura, las dependencias, los componentes, los estilos y el comportamiento actual. Conservá lo que funcione correctamente y adaptalo a esta

especificación. No destruyas funcionalidades existentes ni reconstruyas el proyecto innecesariamente.

Cuando falten decisiones menores, elegí la solución más coherente con los objetivos descritos. Cuando una funcionalidad compleja requiera varias etapas, implementá primero una versión funcional y extensible, sin simular capacidades que todavía no existen.

---

1. IDENTIDAD Y CONCEPTO DEL PRODUCTO

1.1. Nombre

El nombre oficial y definitivo de la aplicación es **Graphia**.

Utilizá Graphia en la interfaz, los títulos, los metadatos, la documentación y cualquier referencia visible al producto.

No utilices nombres anteriores ni introduzcas una marca alternativa.

1.2. Idea principal

Graphia es un editor de documentos matemáticos y científicos que permite escribir, desarrollar y presentar procedimientos paso a paso dentro de un documento continuo.

La aplicación debe permitir combinar libremente:

- Texto normal.
- Expresiones y ecuaciones matemáticas.

- Fracciones, raíces, potencias, integrales, derivadas y sumatorias.
- Símbolos científicos.
- Cuadros de texto.
- Formas geométricas.
- Flechas, vectores y diagramas.
- Diagramas de cuerpo libre para física.
- Gráficos y ejes cartesianos.
- Tablas de datos.
- Representaciones químicas, como estructuras moleculares y reacciones, en una etapa posterior.

El usuario escribe sus propios procedimientos. Graphia no debe resolver automáticamente los ejercicios como comportamiento principal.

El objetivo es proporcionar un entorno donde una persona pueda explicar cómo resolvió un problema, mostrar sus cálculos, realizar dibujos y organizar visualmente sus razonamientos.

1.3. Filosofía del producto

Graphia debe combinar tres experiencias:

1. La libertad de escritura y organización de Google Docs.
2. La precisión y las sugerencias contextuales de Visual Studio Code.
3. La expresividad gráfica de un editor científico.

El documento siempre debe ser el protagonista.

Las herramientas deben aparecer cuando sean necesarias, sin saturar la interfaz. La experiencia debe resultar intuitiva incluso para una persona que nunca haya utilizado un editor matemático especializado.

Principio fundamental: Graphia no piensa ni resuelve en lugar del usuario; le permite expresar sus ideas científicas con precisión.

---

2. DIRECCIÓN VISUAL Y SISTEMA DE DISEÑO

2.1. Estética general

Tomá como referencia el concepto visual de Graphia ya definido: una aplicación de escritorio oscura, elegante, con superficies azul noche, acentos azul eléctrico y violeta, paneles redondeados y una composición moderna.

La referencia es una dirección artística, no una plantilla que haya que reproducir literalmente.

Quiero una interfaz:

- Moderna y sofisticada.
- Minimalista, pero no vacía.
- Redondeada, pero no excesivamente caricaturesca.
- Diferente de los editores convencionales.
- Clara y legible durante sesiones largas.
- Con jerarquía visual definida.
- Con animaciones que aporten significado.
- Centrada en el documento y no en tarjetas decorativas.

Evitá la apariencia de un dashboard administrativo, una plantilla genérica de SaaS o una copia exacta de Notion.

2.2. Paleta de colores

Usá los siguientes valores como punto de partida, adaptándolos si mejora la coherencia visual:

- Fondo principal: `#080D1B`.
- Superficie secundaria: `#10192D`.
- Superficie elevada: `#17223A`.
- Bordes sutiles: `#27344D`.
- Azul de acento: `#718CFF`.
- Violeta de acento: `#9B82FF`.
- Texto principal: `#F0F3FA`.
- Texto secundario: `#A0ADC5`.
- Verde de éxito: `#65D6B0`.
- Rojo de error: `#FF7889`.

Los colores deben definirse mediante variables o tokens centralizados.

No disperses valores de color arbitrarios por los componentes.

El verde debe reservarse principalmente para estados exitosos. Los acentos luminosos no deben estar activos constantemente.

2.3. Esquinas y geometría

La redondez debe formar parte de la identidad de Graphia.

Definí una escala consistente de radios:

- Controles pequeños: aproximadamente 8–12 px.
- Botones y campos: 12–16 px.

- Menús y popovers: 16–20 px.
- Paneles y ventanas flotantes: 20–28 px.
- Elementos destacados: radios más amplios cuando la composición lo justifique.

No apliques el mismo radio indiscriminadamente a todos los componentes.

Los documentos y las áreas de escritura deben conservar una apariencia limpia, sin parecer tarjetas flotantes dentro de otras tarjetas.

2.4. Profundidad y superficies

Utilizá bordes finos, sombras ambientales discretas y diferencias de luminosidad para separar capas.

Podés incorporar transparencias o efectos de cristal de forma selectiva, especialmente en menús flotantes y popovers.

No conviertas toda la aplicación en vidrio translúcido. El área de escritura debe priorizar el contraste y la legibilidad.

2.5. Tipografía

Elegí una familia tipográfica moderna, legible y disponible de manera confiable.

Usá una jerarquía tipográfica consistente para:

- Títulos de documentos.
- Encabezados.
- Texto de escritura.
- Etiquetas de herramientas.

- Metadatos.
- Símbolos matemáticos.
- Estados de guardado.

Las fórmulas deben renderizarse con tipografía matemática adecuada, no como texto convencional con una fuente decorativa.

La tipografía del documento debe poder configurarse independientemente de la tipografía de la interfaz.

2.6. Iconografía

Utilizá iconos vectoriales coherentes, preferentemente de una única biblioteca.

Los iconos deben ser minimalistas y conservar un grosor de trazo uniforme.

Evitá mezclar emojis, iconos de diferentes estilos y símbolos improvisados.

El icono de guardado merece un tratamiento específico, descrito en la sección de animaciones.

---

3. ESTRUCTURA PRINCIPAL DE LA APLICACIÓN

Graphia debe funcionar como una aplicación de documentos, no como un sitio web con páginas promocionales.

3.1. Pantalla de inicio

Implementá una pantalla de inicio donde el usuario pueda:

- Crear un documento nuevo.
- Abrir documentos recientes.
- Buscar documentos.
- Ver documentos favoritos.
- Cambiar el nombre de un documento.
- Duplicar documentos.
- Eliminar documentos.
- Recuperar documentos de una papelera, si se implementa.
- Acceder a la configuración.

Mostrá los documentos mediante una composición visual sobria. Las miniaturas o vistas previas deben aportar información útil, no ser simples tarjetas decorativas.

Incluí estados vacíos que expliquen cómo comenzar.

3.2. Navegación lateral

La barra lateral debe incluir, como mínimo:

- Inicio.
- Todos los documentos.
- Recientes.
- Favoritos.
- Papelera, si está implementada.
- Configuración.

Debe poder contraerse para ampliar el espacio disponible para el documento.

La opción seleccionada debe tener un indicador animado que se desplace entre las opciones.

El estado de navegación debe ser persistente mientras el usuario trabaja.

3.3. Encabezado del editor

El encabezado debe incluir:

- Nombre editable del documento.
- Indicador de guardado.
- Acciones principales del documento.
- Acceso a la configuración o al menú contextual.
- Opciones de exportación cuando estén disponibles.

El nombre del documento debe poder editarse directamente, sin abrir un formulario innecesario.

El indicador de guardado debe ubicarse cerca del título, sin ocupar demasiado espacio.

3.4. Barra de herramientas

La barra debe proporcionar acceso rápido a las acciones de edición más importantes:

- Deshacer y rehacer.
- Estilos de texto.
- Negrita, cursiva y subrayado.

- Encabezados.
- Listas.
- Alineación.
- Tablas.
- Enlaces.
- Inserción de fórmulas.
- Inserción de diagramas.
- Inserción de gráficos.
- Inserción de cuadros de texto.
- Símbolos científicos.

Organizá las herramientas por contexto y frecuencia de uso.

No muestres todos los controles posibles permanentemente. Las acciones avanzadas pueden encontrarse en menús secundarios.

La barra debe ser compacta y permitir que el área de escritura conserve el máximo espacio posible.

3.5. Área de documento

Esta es la superficie principal de Graphia.

Debe ofrecer una experiencia de edición continua, similar a un procesador de texto.

El usuario tiene que poder:

- Escribir texto libremente.
- Insertar fórmulas entre párrafos.
- Crear encabezados y listas.
- Insertar tablas.

- Agregar objetos gráficos.
- Continuar escribiendo después de una fórmula.
- Seleccionar, copiar y pegar contenido.
- Moverse por el documento mediante teclado.
- Deshacer y rehacer operaciones.
- Editar documentos extensos sin degradaciones importantes de rendimiento.

Los objetos matemáticos y gráficos deben integrarse en el flujo del documento, sin obligar al usuario a cambiar constantemente de pantalla.

3.6. Panel de herramientas científicas

Creá un panel contextual de herramientas que pueda abrirse y cerrarse.

Debe incluir categorías como:

- Matemática.
- Física.
- Gráficos.
- Símbolos.
- Plantillas.
- Química, cuando exista una implementación funcional.

El panel debe tener buscador, categorías y accesos rápidos.

Debe poder contraerse para devolver espacio al documento.

No debe comportarse como un agente conversacional ni incluir una sección de asistente de inteligencia artificial.

---

4. EDITOR DE DOCUMENTOS

4.1. Edición de texto

Implementá un editor real, no un `textarea` con botones decorativos.

Debe soportar:

- Escritura y edición.
- Selección de texto.
- Copiar, cortar y pegar.
- Deshacer y rehacer.
- Formato de texto.
- Encabezados.
- Listas ordenadas y desordenadas.
- Enlaces.
- Tablas.
- Saltos de línea.
- Pegado de texto externo.
- Edición de documentos largos.
- Navegación mediante teclado.

Las operaciones de edición deben preservar el contenido y mantener una experiencia de escritura natural.

4.2. Modelo de documento

Diseñá una estructura de documento que permita combinar texto y objetos científicos.

Los elementos deben tener una representación interna clara y serializable.

Por ejemplo:

- Bloques de texto.
- Encabezados.
- Párrafos.
- Fórmulas.
- Tablas.
- Diagramas.
- Gráficos.
- Cuadros de texto.

Cada objeto debe poder editarse después de insertarlo.

Evitá guardar todo el documento como una captura de pantalla, una imagen o una cadena de texto sin estructura.

4.3. Selección y edición de objetos

Los objetos científicos deben poder seleccionarse y mostrar controles contextuales.

Según el tipo de objeto, deben permitir:

- Editar su contenido.
- Cambiar su tamaño.
- Moverlos dentro de los límites de su contenedor.
- Duplicarlos.
- Eliminarlos.
- Cambiar su alineación.

- Editar sus propiedades.

No todos los objetos deben ser libremente posicionables. Las fórmulas en línea deben conservar el comportamiento natural del texto; los diagramas pueden disponer de un lienzo propio.

La interacción debe ser consistente y predecible.

4.4. Diseño adaptable

La experiencia principal debe optimizarse para escritorio, especialmente para resoluciones habituales de portátiles.

En pantallas pequeñas:

- La navegación lateral debe poder ocultarse.
- Las herramientas deben reorganizarse.
- Los paneles flotantes deben adaptarse al espacio disponible.
- Los controles deben conservar tamaños interactivos adecuados.
- El documento debe seguir siendo legible y editable.

No reduzcas simplemente toda la interfaz de escritorio para encajarla en una pantalla móvil.

---

5. SISTEMA DE AUTOCOMPLETADO MATEMÁTICO

Esta es una de las funcionalidades distintivas de Graphia y debe tratarse como una prioridad de producto.

5.1. Concepto

Implementá un sistema inspirado en IntelliSense de Visual Studio Code.

Mientras el usuario escribe, Graphia identifica términos o comandos y presenta sugerencias cerca del cursor.

Las sugerencias deben ser contextuales, rápidas y navegables mediante teclado.

No deben interrumpir la escritura normal.

5.2. Ejemplo principal: raíces

Si el usuario escribe «raíz», debe aparecer un popover debajo del cursor con sugerencias como:

- Raíz cuadrada.
- Raíz cúbica.
- Raíz enésima.

En inglés:

- Square root.
- Cube root.
- Nth root.

La lista debe actualizarse a medida que se escribe.

El usuario debe poder navegar con las flechas del teclado, confirmar con Enter y cerrar con Escape.

También debe poder seleccionar una sugerencia mediante el ratón.

5.3. Inserción estructurada

Cuando el usuario elige «Raíz cuadrada», Graphia debe insertar una estructura matemática editable, no un símbolo Unicode aislado cuando se necesita un radicando.

El cursor debe quedar dentro del espacio editable de la raíz, listo para escribir el contenido.

El usuario debe poder escribir, por ejemplo, `x+1`, y continuar fuera de la raíz sin romper la estructura.

Las estructuras deben admitir navegación por teclado entre sus partes.

Algunas estructuras pueden contener campos editables: numerador, denominador, exponente, índice de raíz, límites de una integral y otros elementos.

5.4. Catálogo de sugerencias

Implementá progresivamente sugerencias para:

Álgebra

- Potencias.
- Subíndices.
- Fracciones.
- Valor absoluto.
- Paréntesis y corchetes.
- Sistemas de ecuaciones.
- Vectores.
- Matrices.

- Determinantes.

Cálculo

- Límites.
- Derivadas.
- Derivadas parciales.
- Integrales definidas e indefinidas.
- Sumatorias.
- Productorias.
- Series.
- Logaritmos.
- Funciones trigonométricas.
- Infinitos.
- Símbolos de pertenencia.
- Conjuntos e intervalos.

Geometría

- Ángulos.
- Triángulos.
- Paralelismo.
- Perpendicularidad.
- Longitudes.
- Coordenadas.
- Símbolos geométricos.

Física

- Vectores.
- Magnitudes.
- Unidades.

- Símbolos griegos.
- Subíndices y superíndices.
- Operadores vectoriales.

Química

En una fase posterior, incorporar fórmulas químicas, subíndices, superíndices de carga, flechas de reacción y símbolos relevantes.

5.5. Comportamiento configurable

En la configuración debe existir una opción para activar o desactivar el autocompletado.

Debe contemplarse la posibilidad de configurar:

- Activación global.
- Idioma de los comandos.
- Comportamiento de las sugerencias.
- Navegación mediante teclado.
- Forma de insertar determinadas estructuras.
- Atajos de teclado, cuando estén implementados.

La preferencia debe persistir entre sesiones.

No implementes ajustes que no tengan efecto real.

5.6. Evitar falsos positivos

Graphia no debe transformar automáticamente cualquier aparición de la palabra «raíz» en una expresión matemática.

El sistema debe distinguir entre texto normal y contexto matemático.

Podés utilizar un delimitador explícito, un modo de fórmula o un sistema de sugerencias que no confirme la conversión hasta que el usuario elija una opción.

La implementación debe priorizar que las sugerencias sean útiles y no intrusivas.

5.7. Arquitectura del autocompletado

Separá el catálogo de comandos de la interfaz visual.

Cada comando debería tener información estructurada como:

- Identificador.
- Etiqueta en español.
- Etiqueta en inglés.
- Alias y términos de búsqueda.
- Categoría.
- Estructura matemática a insertar.
- Posición inicial del cursor.
- Campos editables.
- Atajos opcionales.

No hardcodees toda la lógica en un único componente.

---

6. EDITOR MATEMÁTICO

6.1. Requisito principal

Las fórmulas deben ser objetos matemáticos reales, editables y persistentes.

No basta con mostrar expresiones correctamente renderizadas.

El usuario debe poder entrar en una fórmula, cambiar su contenido, navegar entre sus campos y continuar escribiendo en el documento.

6.2. Estructuras necesarias

Implementá, como mínimo:

- Raíces.
- Fracciones.
- Potencias.
- Subíndices.
- Paréntesis matemáticos.
- Sumatorias.
- Integrales.
- Límites.
- Derivadas.
- Logaritmos.
- Funciones trigonométricas.
- Símbolos griegos.
- Vectores.
- Sistemas de ecuaciones.
- Matrices básicas.

Las estructuras deben representarse correctamente en pantalla y mantenerse editables.

6.3. Librerías posibles

Evaluá MathLive para la edición de fórmulas y KaTeX para la representación matemática cuando corresponda.

No incorpores ambas indiscriminadamente. Definí responsabilidades claras para cada tecnología y verificá su compatibilidad con el editor elegido.

Si una biblioteca ya cubre correctamente una funcionalidad, aprovechala antes de construir una solución paralela.

6.4. Presentación

Las fórmulas deben alinearse con el texto, conservar espacios adecuados y mantener una tipografía matemática legible.

Las expresiones grandes deben adaptarse al ancho del documento.

No permitas que una fórmula larga rompa el diseño general o tape otros elementos.

---

7. DIAGRAMAS, FORMAS Y OBJETOS CIENTÍFICOS

7.1. Sistema de dibujo

Graphia debe incorporar herramientas para representar gráficamente los procedimientos científicos.

El usuario debe poder insertar y editar:

- Rectángulos.
- Cuadrados.
- Círculos.
- Líneas.
- Flechas.
- Vectores.
- Etiquetas.
- Cuadros de texto.
- Ejes cartesianos.
- Conectores.
- Formas geométricas sencillas.

7.2. Edición de formas

Cada objeto debe admitir selección y manipulación.

Según corresponda, debe permitir:

- Cambiar tamaño.
- Cambiar posición.
- Modificar el trazo.
- Modificar el relleno.
- Cambiar la orientación.
- Agregar etiquetas.
- Duplicar.
- Eliminar.
- Agrupar y desagrupar, si se implementa.

Utilizá controles de selección claros y discretos.

No muestres todos los controles permanentemente: aparecen cuando el objeto está seleccionado.

7.3. Diagramas de cuerpo libre

Esta es una funcionalidad importante para física.

El usuario debe poder dibujar un bloque, seleccionar un punto de origen y agregar vectores que representen fuerzas.

Por ejemplo, debe poder representar:

- Peso.
- Fuerza normal.
- Fricción.
- Tensión.
- Fuerza aplicada.

Las etiquetas deben ser editables y los vectores deben poder cambiar de dirección y longitud.

El sistema no debe asumir automáticamente que un diagrama es físicamente correcto. Su función inicial es permitir dibujarlo y organizarlo.

7.4. Ejes cartesianos y gráficos

Implementá herramientas para insertar ejes cartesianos y representar gráficos.

Cuando sea posible, permití introducir una función matemática para dibujar su gráfica.

Mantené diferenciadas la expresión matemática y su representación gráfica.

Las actualizaciones del gráfico deben conservar la expresión original y no modificar silenciosamente el documento.

7.5. Tecnología

SVG puede servir para las formas vectoriales y los diagramas.

Si necesitás interacciones más complejas, evaluá una biblioteca de dibujo compatible con el modelo del documento.

La arquitectura debe permitir guardar las propiedades de cada objeto y reconstruirlo posteriormente.

7.6. Química

La representación química avanzada puede desarrollarse en una fase posterior.

La arquitectura debe dejar espacio para incorporar:

- Fórmulas moleculares.
- Enlaces.
- Estructuras moleculares.
- Cargas.
- Flechas de reacción.
- Ecuaciones químicas.

No presentes estas funcionalidades como terminadas si todavía no están implementadas.

---

8. CUADROS DE TEXTO Y ELEMENTOS INDEPENDIENTES

El usuario debe poder insertar cuadros de texto dentro del documento para separar datos, observaciones, definiciones o resultados.

Cada cuadro debe permitir:

- Escribir texto.
- Cambiar su tamaño.
- Editar su contenido.
- Moverlo dentro de los límites permitidos.
- Cambiar su alineación.
- Duplicarlo.
- Eliminarlo.

Los cuadros deben mantener un estilo coherente con Graphia.

También se deben contemplar bloques destacados para datos importantes, conclusiones y resultados.

No conviertas automáticamente todos los párrafos en tarjetas.

La distinción entre texto continuo y objetos independientes debe ser clara.

---

9. IDIOMAS: ESPAÑOL E INGLÉS

9.1. Idiomas iniciales

Graphia debe ofrecer dos idiomas:

- Español.
- Inglés.

No es necesario implementar alemán, francés, portugués u otros idiomas en la primera versión.

9.2. Interfaz

Todos los textos de la interfaz deben traducirse correctamente:

- Navegación.
- Menús.
- Botones.
- Herramientas.
- Configuración.
- Mensajes.
- Estados de guardado.
- Errores.
- Ayudas contextuales.
- Etiquetas de accesibilidad.

No mezcles ambos idiomas dentro de la misma interfaz salvo que se trate de contenido escrito por el usuario.

9.3. Comandos matemáticos

El catálogo de autocompletado debe reconocer términos en español e inglés.

Por ejemplo, «raíz cuadrada» y «square root» deben acceder a la misma estructura matemática.

La etiqueta presentada puede depender del idioma de los comandos.

9.4. Configuración independiente

Siempre que resulte práctico, permití configurar independientemente:

- Idioma de la interfaz.
- Idioma de los comandos matemáticos.

Esto permite usar la interfaz en español con comandos en inglés, o viceversa.

9.5. Implementación técnica

Centralizá las traducciones en archivos o módulos de internacionalización.

No disperses textos traducibles por toda la aplicación.

La selección de idioma debe persistir entre sesiones.

---

10. SISTEMA DE GUARDADO Y PERSISTENCIA

10.1. Guardado automático

Los documentos deben guardarse automáticamente.

No debe ser necesario pulsar un botón después de cada modificación.

Implementá una estrategia de guardado con debounce o agrupación de cambios para evitar operaciones excesivas.

La interfaz debe distinguir entre:

- Cambios pendientes.
- Guardado en curso.
- Guardado completado.
- Error de guardado.

No muestres un estado de éxito antes de confirmar que los datos se guardaron correctamente.

10.2. Persistencia local

Para una primera versión, priorizá el guardado local mediante IndexedDB u otra solución apropiada.

El contenido debe sobrevivir al cierre y reapertura de la aplicación y a una recarga normal de la página.

Guardá la estructura completa del documento, incluidas las fórmulas y los objetos gráficos.

No guardes solamente el texto visible.

10.3. Gestión de documentos

Implementá operaciones funcionales para:

- Crear.
- Abrir.

- Renombrar.
- Duplicar.
- Eliminar.
- Recuperar, si existe papelera.
- Exportar, cuando esté implementado.

Cada documento debe tener un identificador estable, fechas de creación y modificación, y su contenido estructurado.

10.4. Integridad de los datos

Evitá perder modificaciones cuando se producen cambios rápidos o cuando se cierra una vista.

Controlá errores de persistencia y mostrale al usuario un estado comprensible.

No declares guardado un documento cuyo contenido todavía no se haya persistido.

La sincronización en la nube puede añadirse posteriormente mediante una arquitectura compatible, pero no debe simularse en la primera versión.

---

11. ANIMACIONES Y MICROINTERACCIONES

Las animaciones son parte de la identidad de Graphia.

Deben sentirse fluidas, precisas, elegantes y naturales, con inspiración en las transiciones de macOS y las microinteracciones de Apple.

No quiero animaciones genéricas aplicadas a todo. Cada movimiento debe corresponder a una acción concreta.

11.1. Principios generales

- Mantener la interfaz reactiva.
- Evitar retrasos en la escritura.
- Preservar la posición del cursor.
- Evitar saltos de layout.
- Utilizar aceleraciones y desaceleraciones naturales.
- Aplicar spring solo donde aporte valor.
- Evitar rebotes exagerados.
- No utilizar efectos decorativos constantes.
- Mantener una experiencia accesible.

11.2. Botones

Los botones pueden reducir ligeramente su escala al pulsarse y regresar suavemente a su tamaño original.

La respuesta debe ser inmediata.

Duración orientativa de pulsación: 80–120 ms.

Retorno: 180–280 ms.

No aplicar un rebote fuerte a todos los botones.

11.3. Paneles

Los paneles deben expandirse desde su posición original, conservando una continuidad espacial.

Al abrir herramientas matemáticas, el panel puede combinar un cambio de tamaño con una transición de opacidad.

Duración orientativa: 300–450 ms para expansiones grandes.

Los elementos interiores pueden aparecer con un pequeño retraso escalonado, sin exagerar el efecto.

11.4. Popovers y autocompletado

Las sugerencias deben aparecer cerca del cursor con un desplazamiento mínimo y un fundido rápido.

Duración orientativa: 120–180 ms.

El elemento seleccionado debe cambiar de estado suavemente.

Al confirmar una sugerencia, la inserción de la fórmula debe parecer una continuación de la acción, no una ventana que se cierra y otra que se abre.

11.5. Navegación

El indicador de navegación debe deslizarse entre las opciones seleccionadas.

Duración orientativa: 250–350 ms.

Los cambios de contenido deben acompañar el movimiento sin producir transiciones largas ni bloquear la interacción.

11.6. Inserción de fórmulas

La inserción debe ser visualmente cuidada, pero no puede perjudicar la edición.

Cuando sea técnicamente apropiado, las estructuras pueden expandirse desde su punto de inserción.

Una fracción puede revelar sus campos editables mediante una transición breve.

Una raíz puede mostrar su radicando preparado para escribir.

El cursor debe permanecer en la posición lógica correcta.

11.7. Diagramas

Al dibujar una flecha, su trazo puede extenderse desde el punto inicial hasta el final.

Las etiquetas pueden aparecer después de que se dibuje el elemento.

Al mover un objeto, sus conectores deben acompañarlo de forma continua.

Las animaciones deben representar las acciones reales y no sustituirlas por secuencias decorativas.

11.8. Apertura de documentos

La apertura de un documento puede utilizar una transición breve de continuidad espacial, haciendo que el elemento seleccionado dé paso al editor.

No bloquees la interacción durante la transición.

11.9. Cambio de tema

Los colores pueden interpolarse suavemente al cambiar entre los temas disponibles.

No animes individualmente cientos de elementos si eso produce parpadeos o degradación del rendimiento.

11.10. Implementación

Inspeccioná las tecnologías existentes antes de elegir una solución.

CSS, SVG y la biblioteca de animación ya instalada pueden ser suficientes.

No agregues una dependencia pesada sin justificarla.

Las animaciones deben estar centralizadas y utilizar valores consistentes.

---

12. ANIMACIÓN DE GUARDADO: DISQUETE QUE SE TRANSFORMA EN TICK

Esta interacción es especialmente importante y debe tener una implementación dedicada.

Quiero una microanimación inspirada en la continuidad visual de Face ID de Apple, adaptada a la identidad de Graphia.

12.1. Concepto

El icono de un disquete debe transformarse físicamente en un tick cuando el documento se guarda correctamente.

No quiero que el disquete desaparezca mediante un fundido y que luego aparezca otro icono.

Debe sentirse como un único objeto que cambia de forma.

12.2. Secuencia

Estado 1: guardado pendiente o en curso

Mostrar el disquete en una posición fija, cerca del nombre del documento.

Cuando corresponda, puede realizar una contracción sutil o un pulso breve.

No utilizar un spinner permanente.

Estado 2: transformación

Cuando el guardado se completa con éxito, iniciar la transformación del disquete en un tick.

La silueta debe simplificarse y sus trazos deben convertirse progresivamente en la forma del tick.

El centro visual y la posición del icono deben permanecer estables.

Estado 3: confirmación

El tick termina de dibujarse y se estabiliza con un spring muy sutil.

Se puede añadir un pequeño acento verde menta, sin partículas, destellos intensos ni explosiones visuales.

Estado 4: reposo

El tick permanece brevemente y luego desaparece suavemente o queda como indicador discreto de guardado completado, según el diseño elegido.

12.3. Parámetros

- Duración orientativa de la transformación: 500–700 ms.
- Trazo limpio y consistente.
- Curvas suaves.
- Rebote mínimo.
- Sin cambios de posición del encabezado.
- Sin saltos de layout.
- Sin parpadeos.
- Sin animaciones que bloqueen la escritura.

12.4. Implementación técnica

Preferí SVG en línea con animación de trazos y geometría.

Podés utilizar interpolación de paths compatibles, transiciones coordinadas entre formas o una secuencia de trazos diseñada cuidadosamente.

`stroke-dashoffset` puede servir para dibujar progresivamente el tick, pero no sustituye por sí solo una transformación geométrica real.

No simules toda la animación mediante cambios de opacidad entre dos iconos.

El resultado debe ser visualmente convincente, no una aproximación improvisada.

12.5. Integración con el guardado real

La animación solo puede activarse después de confirmar que el guardado terminó correctamente.

No debe reproducirse por cada pulsación de tecla.

Agrupá las modificaciones y evitá animaciones repetitivas durante la escritura continua.

Si falla el guardado, mostrale al usuario un estado de error y una opción de reintento.

No muestres el tick verde cuando los datos no estén guardados.

Si se produce un nuevo guardado durante la animación, gestioná la transición sin superponer múltiples instancias.

12.6. Accesibilidad

Respetá `prefers-reduced-motion`.

En ese caso, sustituí la transformación por un cambio de estado simple, manteniendo la información de éxito o error.

---

13. CONFIGURACIÓN

Implementá una pantalla de configuración organizada y funcional.

Debe incluir, progresivamente:

General

- Idioma de interfaz.
- Preferencias de apariencia.
- Preferencias de edición.

Escritura

- Activar o desactivar el autocompletado.
- Idioma de los comandos.
- Preferencias de inserción de estructuras.
- Comportamiento del teclado.

Apariencia

- Tema oscuro.
- Tema claro.
- Preferencia del sistema, si se implementa.
- Tamaño de texto del documento.
- Preferencias de movimiento reducido.

Documentos

- Opciones de guardado disponibles.
- Exportación.
- Preferencias de formato, cuando estén implementadas.

Todas las opciones visibles deben funcionar.

Si una preferencia todavía no está implementada, no la presentes como funcional.

Las preferencias deben persistir entre sesiones.

---

14. ACCESIBILIDAD Y EXPERIENCIA DE TECLADO

Graphia debe ser utilizable sin depender exclusivamente del ratón.

Implementá:

- Navegación mediante teclado.
- Estados de foco visibles.
- Etiquetas accesibles para controles.
- Contraste adecuado.
- Estados de error comprensibles.
- Compatibilidad razonable con lectores de pantalla.
- Respeto por las preferencias de movimiento reducido.
- Alternativas de interacción cuando una animación no esté disponible.

Las sugerencias matemáticas deben permitir navegar con flechas, confirmar con Enter y cancelar con Escape.

Los atajos no deben interferir con la escritura ni con las combinaciones habituales del navegador.

Documentá los atajos implementados y evitá mostrar atajos que no funcionen.

---

15. ARQUITECTURA TÉCNICA

Antes de elegir tecnologías definitivas, inspeccioná el proyecto existente.

Si todavía no existe una base de código, elegí una arquitectura mantenible que permita desarrollar el editor progresivamente.

15.1. Frontend

Podés utilizar:

- React.
- TypeScript.
- Vite u otra herramienta apropiada.
- CSS modular o una arquitectura de estilos equivalente.
- Un sistema centralizado de design tokens.

Estas son opciones, no una obligación de reemplazar tecnologías que ya funcionen.

15.2. Editor

Evaluá Tiptap con ProseMirror para el documento estructurado.

Las fórmulas deben integrarse mediante extensiones específicas que permitan editar estructuras matemáticas.

No trates el documento completo como un simple campo de texto.

15.3. Matemática

Evaluá MathLive para la edición de fórmulas y KaTeX para renderizado cuando corresponda.

Verificá compatibilidad, rendimiento y responsabilidades antes de incorporar ambas bibliotecas.

15.4. Diagramas

Evaluá SVG o una biblioteca de dibujo apropiada.

Los diagramas deben guardar sus propiedades y reconstruirse desde los datos almacenados.

15.5. Persistencia

Para la primera versión local, utilizá IndexedDB o una solución equivalente.

Separá la persistencia de la interfaz para que, en el futuro, sea posible incorporar sincronización remota sin reescribir el editor.

15.6. Estructura de carpetas

Organizá el código por responsabilidades.

Como orientación, considerá módulos separados para:

- Aplicación y navegación.
- Editor de documentos.
- Extensiones matemáticas.
- Autocompletado.
- Herramientas científicas.
- Diagramas.
- Persistencia.
- Configuración.
- Internacionalización.
- Sistema de diseño.
- Animaciones.
- Utilidades compartidas.

No conviertas cada pequeño elemento en una abstracción innecesaria, pero evitá los componentes monolíticos que concentren toda la lógica.

15.7. CSS

No utilices CSS inline.

Centralizá las variables visuales, mantené una nomenclatura coherente y evitá reglas globales que interfieran con el editor.

Los estilos deben ser mantenibles y no depender de selectores frágiles.

---

16. RENDIMIENTO Y ESTABILIDAD

El editor debe sentirse rápido durante la escritura y la edición de fórmulas.

Priorizá:

- Actualizaciones eficientes.
- Evitar renders innecesarios.
- Debounce en las operaciones de guardado.
- Gestión estable de la selección del editor.
- Limpieza de listeners y recursos.
- Control de errores.
- Carga diferida de herramientas pesadas cuando tenga sentido.
- Manejo correcto de documentos extensos.

Las animaciones no deben bloquear la escritura.

Los diagramas no deben reconstruirse por completo cada vez que cambia un carácter en el documento.

Las fórmulas deben conservar su estado de edición cuando se actualiza contenido cercano.

No optimices prematuramente a costa de complicar toda la arquitectura, pero evitá decisiones que hagan inviable el crecimiento del proyecto.

---

17. EXPORTACIÓN Y PORTABILIDAD

La exportación debe incorporarse por etapas.

Priorizá:

- Exportar el documento como un formato estructurado que permita recuperar su contenido.
- Exportar a PDF cuando la implementación sea fiable.
- Exportar a HTML, si resulta útil.
- Mantener la representación de fórmulas.
- Preservar diagramas y tablas.
- Evitar perder contenido al importar o exportar.

No ofrezcas botones de exportación que no produzcan archivos válidos.

La exportación PDF debe conservar el orden del documento, la tipografía, las fórmulas y los objetos gráficos.

Si una funcionalidad requiere una dependencia o un proceso adicional, documentalo y mantené una alternativa funcional mientras tanto.

---

18. SEGURIDAD Y FIABILIDAD

Aunque la primera versión sea local, mantené buenas prácticas.

- Validá los datos antes de utilizarlos.
- Evitá insertar HTML arbitrario sin sanitización.
- Tratá el contenido de documentos como datos, no como código ejecutable.
- No expongas secretos en el frontend.
- No incluyas claves o credenciales ficticias.
- Gestioná los errores de almacenamiento.
- Evitá perder documentos ante fallos inesperados.

- No dependas de servicios externos para funciones básicas que deberían funcionar localmente.

Si se incorpora un backend en el futuro, diseñá las interfaces de persistencia para permitir esa evolución.

---

19. PRUEBAS Y CRITERIOS DE ACEPTACIÓN

No consideres terminado un componente simplemente porque se ve bien.

La aplicación debe probarse funcionalmente.

19.1. Editor

Verificá que el usuario pueda escribir, seleccionar, copiar, pegar, deshacer y rehacer sin perder contenido.

19.2. Fórmulas

Verificá que una raíz, una fracción y una integral puedan insertarse, editarse, guardarse y recuperarse.

Comprobá que el cursor quede dentro de la estructura correcta.

19.3. Autocompletado

Escribí «raíz», seleccioná «raíz cuadrada», insertá una expresión, salí de ella y continuá escribiendo texto normal.

Repetí el flujo en inglés.

Comprobá que Escape cierre las sugerencias y que desactivar el autocompletado cambie realmente el comportamiento.

19.4. Diagramas

Creá una forma, agregá un vector, editá su etiqueta, guardá el documento y volvé a abrirlo.

Los objetos deben conservar sus propiedades.

19.5. Persistencia

Creá un documento, modificá su contenido, recargá la aplicación y comprobá que todo se haya conservado.

Simulá un error de guardado y verificá que no aparezca una confirmación falsa.

19.6. Animación de guardado

Verificá que el disquete se transforme en tick únicamente tras un guardado exitoso.

Comprobá que no haya saltos de layout, animaciones duplicadas ni interferencia con la escritura.

19.7. Interfaz

Comprobá diferentes tamaños de ventana, estados vacíos, documentos largos, paneles abiertos y cerrados y cambios de idioma.

19.8. Calidad del código

Ejecutá las comprobaciones disponibles del proyecto: compilación, lint, comprobación de tipos y pruebas automatizadas.

Corregí los errores introducidos por la implementación.

No ocultes errores mediante configuraciones permisivas o desactivando comprobaciones existentes.

---

20. ORDEN DE IMPLEMENTACIÓN

No intentes desarrollar todas las funcionalidades avanzadas simultáneamente.

Trabajá por etapas, manteniendo una aplicación funcional después de cada una.

Etapa 1: Base visual y estructura

- Definir los design tokens.
- Crear la navegación.
- Implementar la pantalla de inicio.
- Crear el encabezado.
- Crear la barra de herramientas.
- Construir el área principal del documento.
- Definir la arquitectura de componentes.

Resultado esperado: una interfaz navegable, coherente y adaptable.

Etapa 2: Editor y persistencia

- Implementar la edición de texto.
- Crear el modelo del documento.
- Implementar guardado automático local.
- Añadir creación, apertura y renombrado.
- Añadir deshacer y rehacer.

Resultado esperado: un editor de documentos que pueda utilizarse de verdad.

Etapa 3: Fórmulas y autocompletado

- Integrar el editor matemático.
- Implementar estructuras editables.
- Crear el catálogo de comandos.
- Implementar el popover contextual.
- Añadir navegación mediante teclado.
- Incorporar español e inglés.

Resultado esperado: escritura matemática funcional y fluida.

Etapa 4: Herramientas gráficas

- Implementar formas y flechas.
- Añadir cuadros de texto.
- Crear diagramas de cuerpo libre.
- Incorporar ejes y gráficos básicos.
- Guardar y recuperar los objetos.

Resultado esperado: documentos que combinen texto, fórmulas y dibujos.

Etapa 5: Animaciones

- Botones.
- Paneles.
- Navegación.
- Autocompletado.
- Inserción de fórmulas.
- Diagramas.
- Transformación del disquete en tick.

Resultado esperado: microinteracciones integradas con la lógica real de la aplicación.

Etapa 6: Configuración y calidad

- Completar preferencias.
- Añadir tema claro, si corresponde.
- Mejorar accesibilidad.
- Verificar persistencia.
- Añadir exportación prioritaria.
- Corregir errores.
- Probar documentos extensos.

Resultado esperado: una primera versión estable y coherente.

Etapa 7: Ampliaciones científicas

- Herramientas químicas.
- Diagramas más avanzados.
- Mejoras de gráficos.
- Funciones de exportación adicionales.

- Posible sincronización en la nube.

Estas funciones no deben retrasar la consolidación de la experiencia principal.

---

21. REGLAS DE IMPLEMENTACIÓN PARA CODEX

Seguí estas reglas durante todo el desarrollo.

1. Inspeccioná primero el código y entendé la arquitectura existente.
2. No reemplaces funcionalidades que ya funcionan sin una razón concreta.
3. No implementes únicamente una representación visual de los controles: cada acción debe funcionar.
4. No agregues un asistente de IA ni una interfaz conversacional como parte del producto principal.
5. No automatices la resolución de ejercicios.
6. No simules guardados, exportaciones ni otras operaciones reales.
7. No utilices CSS inline.
8. No dupliques lógica que pueda centralizarse de forma razonable.
9. No incorpores dependencias innecesarias.
10. No introduzcas animaciones que bloqueen la edición.
11. No uses transiciones de opacidad como sustituto de la transformación real del disquete en tick.
12. No presentes funciones futuras como si ya estuvieran terminadas.

13. No sacrifiques la estabilidad del editor por efectos visuales.
14. Conservá la selección, la posición del cursor y el contenido al actualizar el documento.
15. Mantené los componentes, estilos, tipos y datos organizados.
16. Si una decisión técnica afecta a varias partes del proyecto, evaluá primero sus consecuencias.
17. Si el alcance resulta demasiado grande para una única iteración, completá la etapa actual y documentá el siguiente paso.
18. Verificá los cambios con las herramientas disponibles antes de dar por terminada cada etapa.

No respondas únicamente con una lista de recomendaciones o un plan teórico. Implementá los cambios concretos en el proyecto.

Si tenés acceso al repositorio y al entorno de ejecución, utilizalos para inspeccionar, modificar y probar la aplicación.

---

22. RESULTADO FINAL ESPERADO

Graphia debe sentirse como una aplicación de escritorio premium, construida alrededor de una idea clara: permitir que las personas expresen su razonamiento matemático y científico con la misma naturalidad con la que escriben un documento.

La experiencia debe reunir:

- Un editor continuo.
- Fórmulas realmente editables.
- Autocompletado inteligente.

- Soporte bilingüe.
- Herramientas de dibujo científico.
- Diagramas integrados.
- Cuadros de texto.
- Guardado fiable.
- Una identidad visual única.
- Animaciones precisas.
- Arquitectura extensible.

El usuario debe poder comenzar con un documento en blanco, escribir un enunciado, desarrollar el procedimiento matemático, insertar una raíz mediante autocompletado, dibujar un diagrama de cuerpo libre, agregar etiquetas y guardar todo sin abandonar el documento.

Al terminar, la aplicación no debe parecer una colección de herramientas ensambladas, sino un sistema unificado.

La calidad de Graphia debe notarse tanto en lo que permite hacer como en la forma en que responde a cada interacción.

---

23. PRIMERA ACCIÓN

Antes de comenzar a modificar archivos:

1. Inspeccioná la estructura actual del proyecto.
2. Identificá el framework, las dependencias y las convenciones existentes.
3. Compará la implementación actual con esta especificación.

4. Identificá qué funcionalidades existen, cuáles están incompletas y cuáles deben crearse.
5. Definí la etapa inicial más apropiada.

Después, comenzá la implementación concreta sin intentar construir todas las funcionalidades avanzadas en un solo cambio.

Priorizá una base sólida, una interfaz visualmente excepcional y un editor funcional.

Al finalizar cada etapa, resumí brevemente qué implementaste, qué comprobaste y qué queda pendiente.

No declares una funcionalidad completada hasta haber verificado su comportamiento real.